

Project Development Phase

ComicCraft – AI Comic Story Creator using Gemini Models

Project basis: This document is prepared from the provided ComicCraft project documentation. The development phase follows the documented milestones for core functionality, FastAPI routing, frontend development, and local deployment. *The source describes ComicCraft as a web application using FastAPI, Gemini models, Stable Diffusion, HTML/CSS/Jinja2, and FPDF.*

1. Project Development Overview

The Project Development Phase converts the ComicCraft design and architecture into a working application. The documented workflow focuses on generating a structured comic outline, expanding it into narration and dialogue, creating illustrations, organizing panels, exporting the comic as a PDF, and presenting the result through a FastAPI-powered web interface.

2. Development Objectives

- Implement the core AI-powered comic generation workflow.
- Process user inputs including story prompt, character name, setting, tone, and art style.
- Generate a structured five-panel comic outline.
- Generate detailed narration and character dialogue.
- Generate comic-style illustrations for each panel.
- Build a structured panel layout that combines images and story text.
- Export the completed comic as a multi-page PDF.
- Provide browser-based pages for input, comic preview, and export confirmation.
- Expose API functionality for JSON-based comic generation and image testing.

3. Technology Stack

Component	Technology	Purpose
Web Framework	FastAPI	Routing, request handling, backend processing
Server	Uvicorn	Runs the FastAPI application locally
Frontend	HTML, CSS, Jinja2	User input, dynamic rendering, comic preview
Story Generation	Gemini Flash	Structured five-panel comic outline
Narration & Dialogue	Gemini Pro	Detailed comic narration and character dialogue
Image Generation	Stable Diffusion / Diffusers	Comic-style panel illustrations
PDF Export	FPDF	Multi-page comic PDF generation
Image Processing	Pillow	Image-related processing
ML Runtime Support	PyTorch / Accelerate	Support for Stable Diffusion

		generation
--	--	------------

4. Development of Core Functionalities

4.1 Structured Comic Outline Generation

The function `generate_outline()` in `gemini_flash.py` uses Gemini Flash to generate a structured five-panel outline from the user's story prompt. Each panel contains a panel number, title, scene description, and image-generation prompt. The result is represented as a list of dictionaries, with one dictionary for each panel.

4.2 Comic Story Narration and Dialogue

The function `generate_story()` in `gemini_pro.py` expands the panel outlines into a full comic-style story. It generates engaging narration and character dialogue for the panels and returns the resulting story as formatted text.

4.3 Comic Illustration Generation

The function `generate_image()` in `image_generator.py` uses Stable Diffusion to generate a comic-style illustration from an image prompt. The generated image is saved under `static/panels` after the prompt is sanitized for safe filenames, and the function returns the saved image path.

4.4 Comic Layout Construction

The `build_comic_layout()` function in `layout_builder.py` combines generated images with the corresponding panel stories. It produces a structured layout containing the panel number, image path, and text.

4.5 PDF Export

The `save_pdf()` function in `exporters.py` compiles the generated comic into a multi-page PDF using FPDF. Each panel's image and narration are placed on separate pages, and the resulting PDF is stored in `static/exports`.

5. FastAPI Backend Development

The backend coordinates user input, AI generation, layout construction, template rendering, and PDF export. Routing logic is handled in `routes.py`.

5.1 Main Routes

Route	Function	Purpose
/	Homepage	Loads <code>index.html</code> and collects comic details.
/generate	Form-based generation	Processes form input and generates the comic preview.
/generate-comic/json	JSON API	Accepts JSON input and returns layout data and PDF path.
/export-success	Export confirmation	Displays successful comic export confirmation.
/test-image	Image test	Tests image generation directly from a custom prompt.

5.2 User Input Processing

- Story prompt
- Main character name
- Setting
- Story tone
- Art style

Form-based requests are captured using FastAPI Form(...) parameters. JSON-based requests use the documented PromptRequest Pydantic schema for validation.

5.3 Comic Generation Pipeline

1. Receive the user's story details.
2. Call generate_outline() to create the five-panel structure.
3. Call generate_story() to create narration and dialogue.
4. Call generate_image() for each panel.
5. Call build_comic_layout() to associate images and story content.
6. Call save_pdf() to create the downloadable comic.
7. Render the comic preview or return the generated layout through the API.

6. Frontend Development

The frontend is implemented with HTML, CSS, and Jinja2 templates. The documented templates are designed to provide a direct flow from comic input to preview and export confirmation.

6.1 index.html

- Collects the story prompt.
- Collects the main character name.
- Collects the setting.
- Collects the story tone.
- Collects the preferred art style.
- Submits the form to the /generate route using POST.

6.2 comic_preview.html

- Displays the generated comic panel-by-panel.
- Shows the panel number and title.
- Displays the generated illustration.
- Shows the scene description.
- Displays captions and narration.
- Uses Jinja2 looping to dynamically render the panel layout.

6.3 export_success.html

- Displays a successful export message.
- Provides a way to return to the homepage and create another comic.

7. Jinja2 Dynamic Rendering

FastAPI's Jinja2Templates is used to dynamically bind backend-generated data to HTML pages. The / route renders index.html, /generate renders comic_preview.html, and /export-success renders export_success.html. This allows generated panel information, images, captions, and narration to be inserted dynamically after user input.

8. Development Environment and Dependencies

The documentation specifies a Python virtual environment and the required libraries for the ComicCraft application.

Example Windows setup:

```
python -m venv comiccraft-env
comiccraft-env\Scripts\activate
pip install fastapi uvicorn jinja2 python-multipart google-generativeai diffusers transformers fpdf
Pillow accelerate
uvicorn app.main:app --reload
```

9. Environment Variables

Sensitive credentials are stored as environment variables. The documentation specifies variables for the Gemini API key and Hugging Face API token.

```
GEMINI_API_KEY=your_gemini_api_key_here
```

```
HF_API_KEY=your-huggingface-api-key-here
```

10. Local Deployment and Verification

8. Create and activate the project virtual environment.
9. Install the dependencies from requirements.txt.
10. Configure the Gemini API key and Hugging Face API token.
11. Start the FastAPI server using Uvicorn.
12. Open the ComicCraft homepage at <http://127.0.0.1:8000>.
13. Use the FastAPI documentation at <http://127.0.0.1:8000/docs> to test API endpoints.

The documented deployment verification checks the complete workflow: AI text generation, image creation, layout building, and PDF export.

11. Expected User Workflow

14. Open the ComicCraft homepage.

15. Enter a story prompt.
16. Provide the main character name, setting, tone, and art style.
17. Submit the form.
18. Wait for the AI pipeline to generate the comic.
19. Review the five generated panels on the Comic Preview page.
20. Download the comic as a PDF.
21. View the export success page and optionally create another comic.

12. Development Outcome

The completed development phase provides an end-to-end comic creation workflow. The application accepts a personalized prompt, generates a structured story outline and detailed narration, creates corresponding illustrations, assembles the content into a panel layout, and exports the result as a PDF. The documented architecture also separates frontend, backend, AI integration, layout, and export responsibilities, supporting future enhancements.

13. Future Enhancement Scope

- User profiles and account management.
- Comic libraries for saving generated comics.
- Multi-page story arcs.
- Additional customization and generation options.

These enhancements are identified in the source documentation as possible future extensions; they are not described as implemented features of the current version.

14. Conclusion

The Project Development Phase establishes the working implementation of ComicCraft by integrating FastAPI, Gemini Flash, Gemini Pro, Stable Diffusion, Jinja2, and FPDF into one workflow. The result is a web-based system that transforms a user's creative idea into a personalized comic with structured panels, narration, illustrations, and a downloadable PDF.